

Working with Git - Local Repo

Working with Git commands

Important Git local Repository Management Commands in snapshot.

Creating and Cloning Repositories

```
# Initialize a new repository or Setup Git in development (working) folder
git init
# Clone an existing repository
git clone https://github.com/username/repository.git
# Clone to a specific folder
git clone https://github.com/username/repository.git custom-folder-name
```

Tracking Changes

```
git status //Check repository status
git add filename.txt // Add specific file
git add . // Add files to staging area (all changes in folder)
git add *.js // Add all JavaScript files
git add src/ // Add entire directory
git commit -m "Add user authentication feature" // Commit changes
git commit --amend -m "Updated message" // Amend the last commit
```

Viewing History

```
git log // View commit history
```

Important Git Branch Management Commands in snapshot.

Branching is where Git's power truly shines, allowing parallel development streams.

Branch Management

```
git branch // Local branches
git branch -r // Remote branches
git branch -a // All branches
git branch feature-login // Create a new branch
git checkout -b feature-payment # Create and switch to a new branch
git checkout main // Switch branches
git branch -m old-name new-name // Rename a branch
git branch -d feature-complete // Delete a branch
git branch -D feature-broken // Force delete
```

Syncing with Remotes

```
git push origin feature-branch // Upload local changes
git branch --set-upstream-to=origin/main main // Set up branch tracking
```

Git workflow

1. Navigate to the directory where you want to use Git

```
$ mkdir test , $cd test
```

2. Initialize new repository

When working with Git, people often mention Git repository, short form as repo. Earlier we installed Git, that doesn't mean we can use it. We need to manually tell machines when to use Git and in which directory. We need to create a Git repository in the directory where we need to use Git. So, Git starts saving history in the directory after Git repo is set up.

Use git init to create a new git repository. Before we can do anything git-related, we must initialize a repo first!. This is something you do once per project. Initialize the repo in the top-level folder containing your project. Git tracks the directory and **all nested directories**.

More information on Git init command can be found at www.git-scm.com/docs/git-init

```
$ git init
```

3. Create feature branch

```
$ git checkout -b feature-payment
```

4. Make changes in initialized folder and check status

Git is Watching Us after init command. Try making a new file in your repo and then run git status again. You'll see that git noticed the file, but it is "untracked. gives information on the current status of a git repository and its contents.

More information on Git init command can be found at www.git-scm.com/docs/git-status

```
$ git status
```

5. Stage the changes

```
$ git add . // add dot says that group all the changes done in current and nested project directory.
```

6. Create checkpoint

"Commit" command will add check point to the history saved in ". Git" folder with a message to refer.

```
$ git commit "your message for checkpoint reference."
```

7. Push changes to remote branch

```
$ git push <remote> <branch>
```